

What Makes Logic Constructive?

A dependency-based view of constructivity

Valeria de Paiva



Chapman University,
March 2026

Thanks for the invitation!



Why Ask This?

- In 2023 I gave an OCIE talk about Dialectica categories, math
- In 2024 I talked here about "Network Mathematics", AI tools
- Today I want to talk to you about more logic, constructive logic.

My aim is to show that constructivity in proof theory is best understood as a *structural property of proofs*.

Not as a syntactic restriction on the number of conclusions in a sequent calculus, nor as an artefact of a term assignment system.

(not-so-hidden agenda: discuss 'good' proof systems for CAL)

The Usual Story

- Intuitionistic logic tends to be equated with single-conclusion sequent calculus (e.g. pg 40 of the new Miller book)
- Single-conclusion sequents provide no weakening or contraction on the right-hand side of a multiple-conclusion sequent, enough to guarantee some notion of construction
- Sometimes Intuitionistic logic = Curry–Howard terms
Example FILL in Hyland&dePaiva, 1993. this is enough too!
- Question: What is really the structural core?

Key old idea:

Logic of proofs, not truths.

Thesis of this work

Constructivity does **not** require:

- single-conclusion restriction
- or full type-theoretic machinery

It requires only:

- Explicit control of dependency between assumptions and conclusions.
- Hence, some annotation on formulas
- Rules that enforce dependencies

Based on *A formulation of linear logic based on dependency-relations* with Torben Brauner, (1997) and *A short note on intuitionistic propositional logic with multiple conclusions*, with Luiz Carlos Pereira (2005)

Multiple conclusion systems

Classically since Gentzen:

$$\Gamma \vdash \Delta$$

- Assumptions may contribute to several conclusions
- Dependency is implicit
- Discharge is global

Where does constructivity fail?

The Minimal Move

Annotate conclusions with dependencies:

$$i : \Gamma \vdash S_1 : A_1, S_2 : A_2 \dots$$

Each conclusion records:

- Which assumptions it depends on via their indices

No restriction on the number of conclusions.

Examples

An easy example:

$$\frac{1 : A \vdash [1] : A \quad 2 : B \vdash [2] : B}{1 : A, 2 : B \vdash [1, 2] : A \wedge B}$$

Another easy one:

$$\frac{1 : A \vdash [1] : A}{\vdash [] : A \rightarrow A}$$

Examples

A not-so-easy one:

$$\begin{array}{c}
 1 : A \vdash [1] : A \\
 \hline
 1 : A, 2 : B \vdash [1, 2] : A \\
 \hline
 1 : A \vdash [1] : (B \rightarrow A) \\
 \hline
 \vdash [] : A \rightarrow (B \rightarrow A)
 \end{array}$$

Worked Derivation in FIL

Goal:

$$\vdash A \rightarrow (B \rightarrow A)$$

Worked Derivation in FIL

Goal:

$$\vdash A \rightarrow (B \rightarrow A)$$

1. Start with identity:

$$n : A \vdash [n] : A$$

Worked Derivation in FIL

Goal:

$$\vdash A \rightarrow (B \rightarrow A)$$

1. Start with identity:

$$n : A \vdash [n] : A$$

2. Add B by weakening with new index m :

$$n : A, m : B \vdash [n, m] : A$$

Worked Derivation in FIL

Goal:

$$\vdash A \rightarrow (B \rightarrow A)$$

1. Start with identity:

$$n : A \vdash [n] : A$$

2. Add B by weakening with new index m :

$$n : A, m : B \vdash [n, m] : A$$

3. Introduce $\rightarrow$ (discharge B , by removing its index m):

$$n : A \vdash [n] : (B \rightarrow A)$$

Worked Derivation in FIL

Goal:

$$\vdash A \rightarrow (B \rightarrow A)$$

1. Start with identity:

$$n : A \vdash [n] : A$$

2. Add B by weakening with new index m :

$$n : A, m : B \vdash [n, m] : A$$

3. Introduce $\rightarrow$ (discharge B , by removing its index m):

$$n : A \vdash [n] : (B \rightarrow A)$$

4. Introduce outer $\rightarrow$ (discharge A by removing its index n):

$$\vdash [] : A \rightarrow (B \rightarrow A)$$

Worked Derivation in FIL

Goal:

$$\vdash A \rightarrow (B \rightarrow A)$$

1. Start with identity:

$$n : A \vdash [n] : A$$

2. Add B by weakening with new index m :

$$n : A, m : B \vdash [n, m] : A$$

3. Introduce $\rightarrow$ (discharge B , by removing its index m):

$$n : A \vdash [n] : (B \rightarrow A)$$

4. Introduce outer $\rightarrow$ (discharge A by removing its index n):

$$\vdash [] : A \rightarrow (B \rightarrow A)$$

Dependency set ends empty: the theorem depends on no assumptions. But care is needed when dealing with indices and sets of dependencies. See paper for full conditions on rules.

Implication as Dependency Control

$$\frac{\Gamma, n : A \vdash [S_i] : B, \Delta}{\Gamma \vdash [S] : A \rightarrow B, \Delta}$$

Key idea:

- B ‘genuinely’ depends on A
- Discharge is locally controlled
- Require that $n \in S_i$ and most importantly that for every other S' in Δ , n is not in S' .
- If the restriction is satisfied, it means that no other formula in Δ depends on the indicated occurrence of A .

Implication is a primary locus of constructivity.

Theorem FIL

Theorem (cut elimination)

If Π is a derivation of $\Gamma \vdash \Delta$ in FIL, then Π can be transformed into a cut-free proof Π' of $\Gamma \vdash \Delta$.

See *A short note on intuitionistic propositional logic with multiple conclusions*, Pereira and de Paiva, 2005.

Comparison

	Classical	Dependency-based
Multiple conclusions	Yes	Yes
Single-conclusion restriction	No	Not needed
Dependency tracking	Implicit	Explicit
Constructive	No	Yes

FIL and FILL

Joint work on:

- Full Intuitionistic Logic (FIL) – Luiz Carlos Pereira (1994,2005)
- Full Intuitionistic Linear Logic (FILL) – Torben Braüner (1997)

Result:

- Multiple-conclusion systems
- Fully intuitionistic
- Constructivity via dependency control
- dependency control does not require full type theory

More systems like this? Yes!

Universal Quantification

Can add classical quantification to FIL:

$$\frac{\Gamma \vdash A(x)}{\Gamma \vdash \forall x A(x)}$$

End up in Intermediary logic.

Cut-elimination theorem for the Logic of Constant Domains,
Kashima and Shimura 1991,1994.

Logic of Constant Domains = Classical quantifier rules over a FIL basis. (they call their dependencies 'connections', maybe it's better?)

Why bother?

What Is Not Required

We do not require:

- Explicit lambda terms (but indices and sets of indices)
- Elaborate proof-term syntax (α -conversion, etc..)
- Strong normalization machinery

These are all legitimate design choices

– some future work

but not the structural essence.

Relation to Curry–Howard

Dependency annotations can be seen as:

A proof-theoretic abstraction/trace of variable dependence in type theory.

Curry–Howard:

- Makes dependency computational

Dependency systems:

- Make it structural

Normalization-Stable Invariant

Under normalization:

- Proof terms change
- Derivations permute
- But dependency structure is preserved

Dependency isolates a structural invariant of proofs.

Why Single-Conclusion Is Too Crude

Single-conclusion systems enforce constructivity by:

- Cardinality restriction

But cardinality is accidental.

Dependency control targets the real structural constraint.

Linear Logic Case

From *A Formulation of Linear Logic based on Dependency Relations*, with Brauner (CSL 1997)

In linear logic:

- Assumptions are resource-sensitive
- Structural rules are controlled

Dependency tracking **complements** resource control.

Constructivity remains structural.

(counterintuitive that it works for a non-linear system?)

The Linear Theorem

A Formulation of Linear Logic based on Dependency Relations

Theorem 17. *Given a proof τ of $\Gamma \vdash \Delta$, we can construct a cut-free proof σ of the same sequent such that*

- *if τ is a proof in FILL then the end-sequent of σ has fewer dependencies than the end-sequent of τ ;*
- *if τ is a proof in FILL then σ is a proof in FILL too.*

Proof. Iteration of Lemma 16. □

Important remark:

Remark. To explain the different flavour of the FILL implication right rule when compared to the Classical Linear Logic one, we could extend (a little) the Brouwer-Heyting-Kolmogorov interpretation of Intuitionistic Logic to say that a proof of $\Gamma \vdash \Delta$ is a function that maps a proof of each formula in Γ to a proof of one of the formulae in Δ together with a specification of which of the formulae is actually proved. The intuitionistic interpretation of a proof of a formula $B \multimap C$ is a function that maps a proof of B into a proof of C . So given a proof τ of the sequent $\Gamma, B \vdash C, \Delta$ and a proof of each formula in Γ , we get a function that maps a proof of B into a proof of either C or a formula in Δ . By imposing the condition that none of the formulae occurring in Δ are allowed to depend on B in τ , we can assume that our function has the property that either we get a proof of C for any proof of B , or we get a proof of a formula in Δ for any proof of B . This enables us to define a function that maps a proof of each formula in Γ into either a proof of $B \multimap C$ or a proof of a formula in Δ .

BHK+ Remark

- We can extend (a little) the Brouwer Heyting Kolmogorov interpretation of IL using our system
- Say that a proof of $\Gamma \vdash \Delta$ is a function that maps a proof of each formula in Γ to a proof of one of the formulae in Δ , together with a specification of which formulae is actually proved.
- The intuitionistic interpretation of a proof of a formula $B \multimap C$ is a function that maps a proof of B into a proof of C
- Given a proof of the sequent $\Gamma, B \vdash C, \Delta$ and a proof of each formula in Γ we get a function that maps a proof of B into a proof of either C or a formula in Δ
- This enables us to define a function that maps a proof of each formula in Γ into either a proof of $B \multimap C$ or a proof of a formula in Δ

Classical vs Intuitionistic

Classical reasoning:

- Unrestricted merging of dependencies

Intuitionistic reasoning:

- Controlled discharge
- Explicit dependency tracking

The difference is structural, not numerical.

Toward Dependent Type Theory

Here:

- Dependencies between propositions

But they echo:

- Dependencies in genuinely dependent type theory

Future direction:

Lift propositional dependency to type-level dependency.

Big Picture

Constructivity is not about:

- Number of conclusions
- Ornamented proof terms

It is about:

Disciplined control of dependency.

Attempting Excluded Middle

Goal: $\vdash A \vee (A \rightarrow \perp)$

Attempting Excluded Middle

Goal: $\vdash A \vee (A \rightarrow \perp)$

Suppose we attempt to construct the right disjunct by introducing the implication. We would need: $A \vdash A, \perp$ and then discharge A into $\perp$.

Attempting Excluded Middle

Goal: $\vdash A \vee (A \rightarrow \perp)$

Suppose we attempt to construct the right disjunct by introducing the implication. We would need: $A \vdash A, \perp$ and then discharge A into $\perp$. In order to produce the left disjunct A , we would require:

$n : A \vdash [n] : A$

Attempting Excluded Middle

Goal: $\vdash A \vee (A \rightarrow \perp)$

Suppose we attempt to construct the right disjunct by introducing the implication. We would need: $A \vdash A, \perp$ and then discharge A into $\perp$. In order to produce the left disjunct A , we would require:

$n : A \vdash [n] : A$

Thus the sequent would have the shape:

$$n : A \vdash [n] : A, [] : \perp$$

Attempting Excluded Middle

Goal: $\vdash A \vee (A \rightarrow \perp)$

Suppose we attempt to construct the right disjunct by introducing the implication. We would need: $A \vdash A, \perp$ and then discharge A into $\perp$. In order to produce the left disjunct A , we would require:

$n : A \vdash [n] : A$

Thus the sequent would have the shape:

$$n : A \vdash [n] : A, [] : \perp$$

Now try to apply $\rightarrow$ -right to form $A \rightarrow \perp$.

Side condition fails:

$$n \in \text{Dep}(A)$$

Attempting Excluded Middle

Goal: $\vdash A \vee (A \rightarrow \perp)$

Suppose we attempt to construct the right disjunct by introducing the implication. We would need: $A \vdash A, \perp$ and then discharge A into $\perp$. In order to produce the left disjunct A , we would require:

$n : A \vdash [n] : A$

Thus the sequent would have the shape:

$$n : A \vdash [n] : A, [] : \perp$$

Now try to apply $\rightarrow$ -right to form $A \rightarrow \perp$.

Side condition fails:

$$n \in \text{Dep}(A)$$

Hence A cannot be discharged.

Excluded middle is blocked by dependency discipline.

FILL: Linear Implication ($\multimap$ -Right)

Sequents:

$$\Gamma \vdash [S_i] : B, \Theta$$

where Γ, Θ are *linear* multisets of resources.

$\multimap$ -**Right**

$$\frac{\Gamma, j : A \vdash [S_i] : B, [...] : \Theta}{\Gamma \vdash (A \multimap B)[S_i \setminus j], \Theta}$$

Side condition:

$$j \notin \text{Dep}(\Theta)$$

That is, A must not occur in the dependency sets of the other conclusions.

Discharge is:

- Resource-sensitive (linear use of A)
- Dependency-local (no cross-contamination)

One-Line Obstruction in FILL

Attempt to discharge A in:

$$i : A \vdash [i] : A, [S_j] : \perp$$

using $\multimap$ -Right.

Side condition requires:

$$i \notin \text{Dep}(A)$$

But:

$$\text{Dep}(A) = \{A\}$$

Discharge blocked.

Dependency isolation prevents linear multiplicative excluded middle.

Contrast: Classical MLL

In classical multiplicative linear logic (MLL), sequents have the form:

$$\Gamma \vdash \Delta$$

Linear implication (via duality with $\wp$) has no dependency side condition.

From:

$$\Gamma, A \vdash B, \Theta$$

one may always infer:

$$\Gamma \vdash A \multimap B, \Theta$$

There is no requirement that A be absent from the other conclusions.

Dependencies may merge freely.

Categorical Perspective: Factorization

Classical MLL is interpreted in a **-autonomous category*.

A sequent

$$\Gamma, A \vdash B, \Theta$$

corresponds to a morphism:

$$f : \Gamma \otimes A \rightarrow B \wp \Theta$$

Forming $A \multimap B$ amounts to factoring:

$$\Gamma \rightarrow (A \multimap B) \wp \Theta$$

In classical MLL:

Such factorizations are always permitted.

Dependencies may merge freely.

Dependency as Controlled Factorization

In FILL, dependency annotations enforce:

A must not contribute to Θ .

Categorically, this means:

The morphism

$$f : \Gamma \otimes A \rightarrow B \wp \Theta$$

may factor through $A \multimap B$ *only if* the component into Θ is independent of A .

No illegitimate partial currying.

Constructivity = controlled factorization.

Big picture

- FIL and FILL are multiconclusion intuitionistic calculi satisfying cut elimination.
- no need for a type theory for their definition (Clouston et al)
- A notion of dependency is enough
- A similar notion of dependency also works for CD, the logic of constant domains (Kashima & Shimura)
- A similar notion also allows for defining a cut-free S5 calculus and for hybrid modal logic (Brauner)

But:

- still more complicated than 'naked' sequent calculus
- worth the trouble?

Historical detour

- FILL arises from the categorical model GC a Dialectica category, Girard's suggestion for CLL
- There are many variants of LL besides ILL and CLL. Light Linear Logic, Bounded LL, Elementary LL, Soft LL (complexity kinds)
- dependency here: A technical solution to how to prove cut-elimination for FILL
- But idea can be adapted to several systems, different logics. Not just a trick?

Philosophical Musings

- A large collection of logics and proof systems
- Encyclopedia of Proof Systems
<https://github.com/ProofSystem/Encyclopedia>
- How to choose 'good' ones?
- Want systems that can afford a Curry-Howard correspondence, how to discover them?
- Do they justify the effort spent?
- Start from families in CAL (Categories, ALgebras, Logics)?

References

- 1 *Full Intuitionistic Linear Logic*, with Hyland, 1993
- 2 *A formulation of linear logic based on dependency-relations*, with Torben Brauner, 1997
- 3 *A short note on intuitionistic propositional logic with multiple conclusions*, with Luiz Carlos Pereira (2005)
- 4 *Multiple conclusion linear logic: cut elimination and more*, with Harley Eades II, 2020.
- 5 *Annotation-free sequent calculi for full intuitionistic linear logic*, Clouston, Ranald and Dawson, Jeremy and Goré, Rajeev and Tiu, Alwen, CSL 2013.
- 6 *Cut-Elimination Theorem for the Logic of Constant Domains*, Kashima, Ryo and Shimura, Tatsuya, Mathematical Logic Quarterly, 1994.